

Artificial Intelligence Fundamentals

SEARCH IN GAMES

Competitive games

Two-players zero-sum game: **adversarial search**

- Players play in turns
- Players have conflicting goals
- Zero-sum game: you win, I lose
- **Perfect information = fully observable**

Initial state s_0
TO-MOVE(s) = player to move in state s
ACTIONS(s) = legal moves in state s
RESULT(s, a) = transition model, resulting in new state
IS-TERMINAL(s) = is the game over or not
UTILITY(s; p) = utility/payoff function gained on a **terminal** state s by player p

Some games

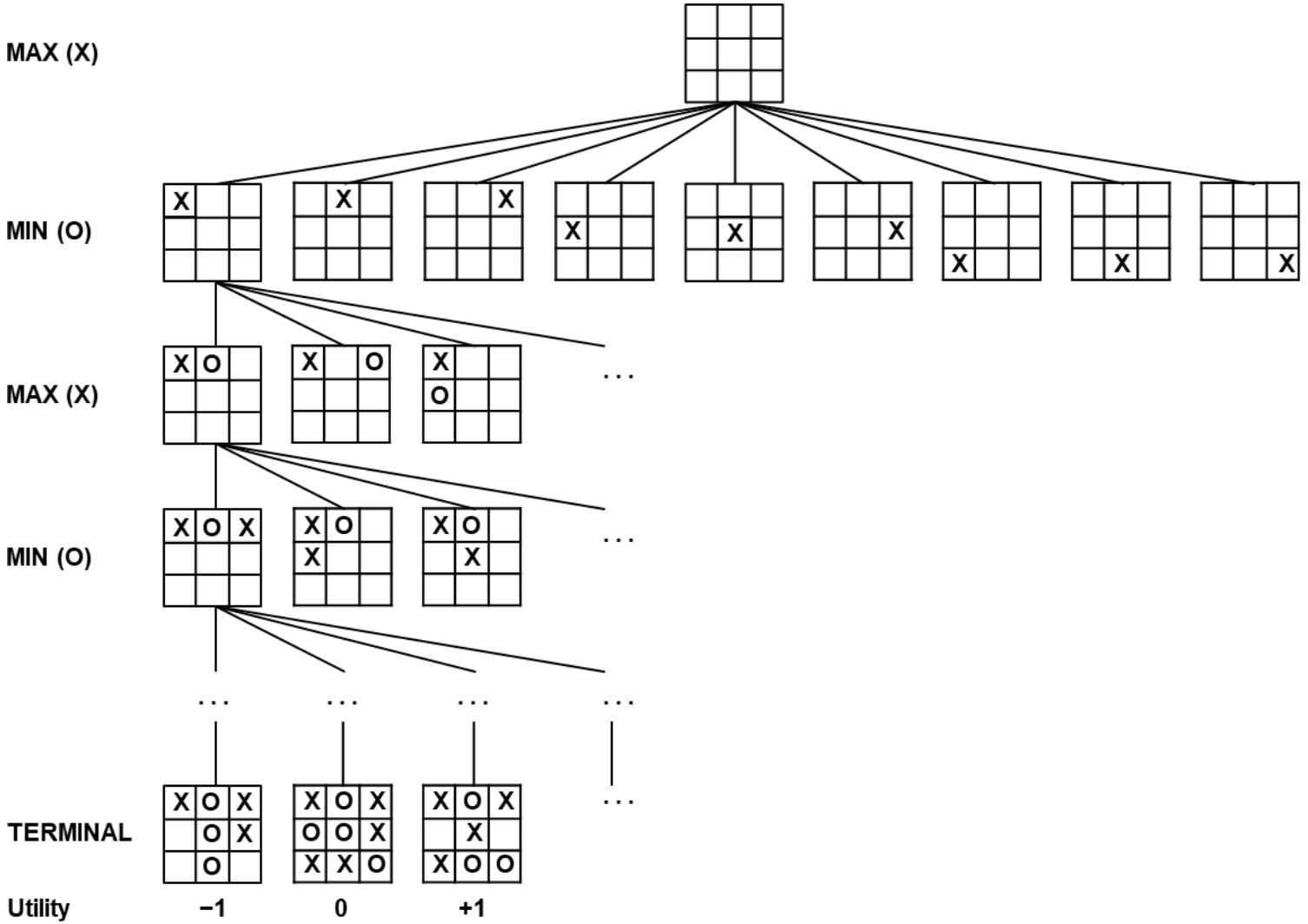
	deterministic	chance-stochastic
perfect information	**chess, checkers, go, othello**	**backgammon monopoly**
imperfect information	**battleships, blind tictactoe**	**bridge, poker, scrabble nuclear war**

Let me introduce you to our players

MAX and MIN

MAX plays first

The utility value is measured from MAX’s perspective



Game tree

A SEARCH TREE THAT FOLLOWS EVERY POSSIBLE SEQUENCE OF MOVES TO A TERMINAL STATES

Game tree

Same properties as search trees you are used to

- Can be infinite if the state space is infinite
- Can be infinite if the state space is finite but allows for repeated states (positions)

No hope to build a full game tree for any reasonable sized games

- < 9! states for tictactoe ≈ 360 000 states
- $\Omega(10^{40})$ states for chess, Go is even larger…



MAX node, where is MAX's turn to move



MIN node, where is MIN's turn to move

Minimax

The strategy for each player is a **conditional plan**

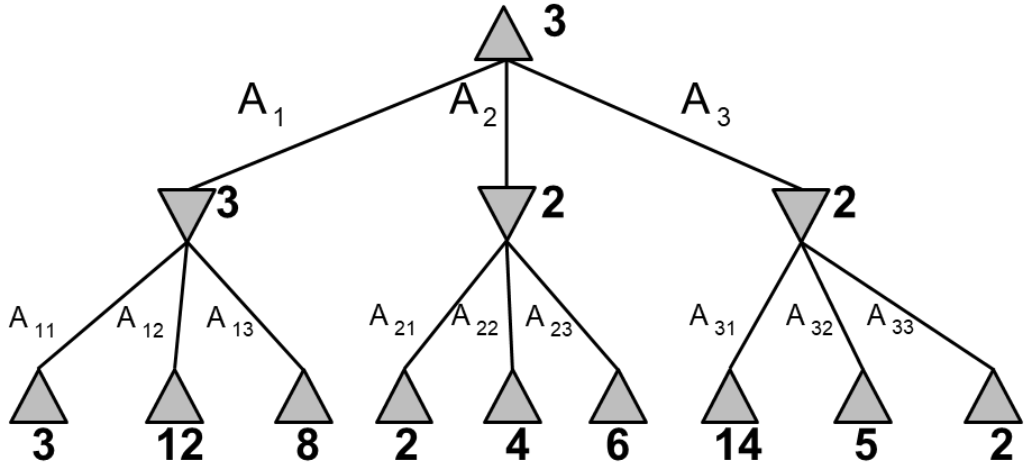
- What should MAX do if MIN does that?

AND/OR search, yet again

- MAX's move = OR node
- MIN's move = AND node

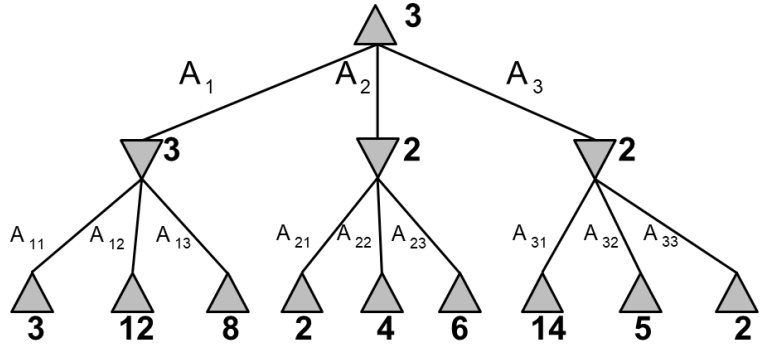
Minimax search (slightly different tree notation)

- **Perfect play** for deterministic, perfect-information games!



Note: a move by one player is called ply since a move often considers a turn made by both players

Minimax value



Once you get the minimax value for each node you can recover the optimal strategy

At leaves (terminal states): MINIMAX(s) = UTILITY(s; p)
- the utility value, aka the final result

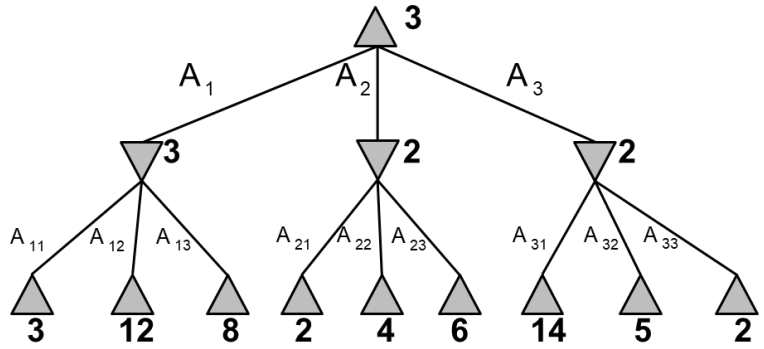
At intermediate nodes (non-terminal states), MINIMAX(s) = the utility value **for MAX**
- assuming that **best play** occurs from that node onwards

MINIMAX(s) = $\max_{a \in ACTIONS(s)} MINIMAX(RESULT(s,a))$, TO-MOVE(s) = MAX

MINIMAX(s) = $\min_{a \in ACTIONS(s)} MINIMAX\big(RESULT(s,a)\big)$, TO-MOVE(s) = MIN

MAX chooses the action that maximizes MINIMAX
MIN chooses the action that minimizes MINIMAX

Minimax search



From the root, perform a complete depth-first search to find all the terminal states (leaves)

Get utility of the leaves

Backtrack to compute MINIMAX of all intermediate nodes

- take the minimum MINIMAX value among all children if the parent is a MIN node, the maximum if it is a MAX node

Complete? Yes, on finite trees

Optimal? Yes, against perfect play

Time complexity? $O(b^m)$

Space complexity? $O(bm)$ = depth-first

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move ← v2, a
    return v, move
```

Minimax algorithm

Playing against suboptimal opponent

Minimax has a slight «inconvenience»: it assumes perfect play

If you play chess against Carlsen, you should probably always play the best move

- And you will lose

If you play against me, you are not expecting perfect play.
If the position is a draw with perfect play, you might want to do a tricky, but suboptimal move to try and take the win. Minimax will never do that.

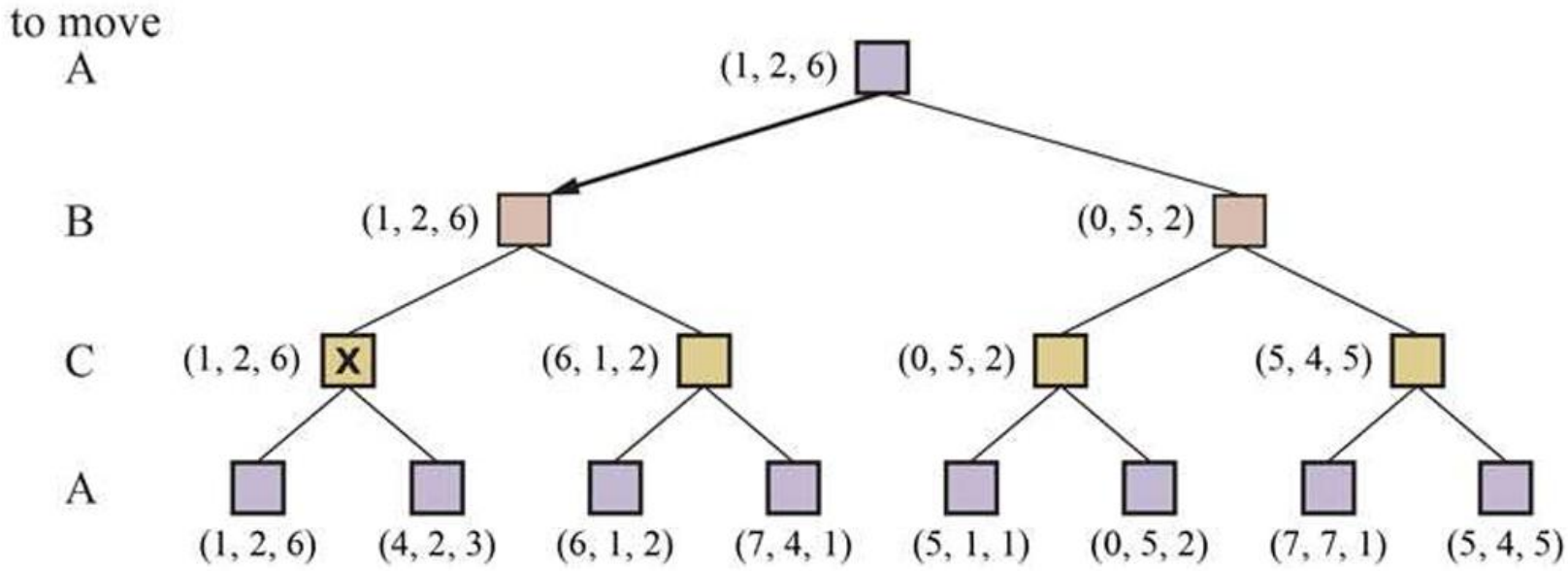
- If I play the only correct move in the position: you lose (p=1%)
- If I play some other moves: you win (p=85%)
- If I play anything else: it's a draw (p=14%)

This is the difference between the so-called «computer moves» and «human moves»

- There are computer programs that try to imitate how humans behave against suboptimal opponents

Extension to >2 players

Each node N chooses the successor with the largest evaluation associated with the player at node N



The first three ply of a game tree with three players (*A*, *B*, *C*). Each node is labeled with values from the viewpoint of each player. The best move is marked at the root.

Practical complexity of minimax

Branching factor in chess = 35

Average game length = 40 moves = 80 ply

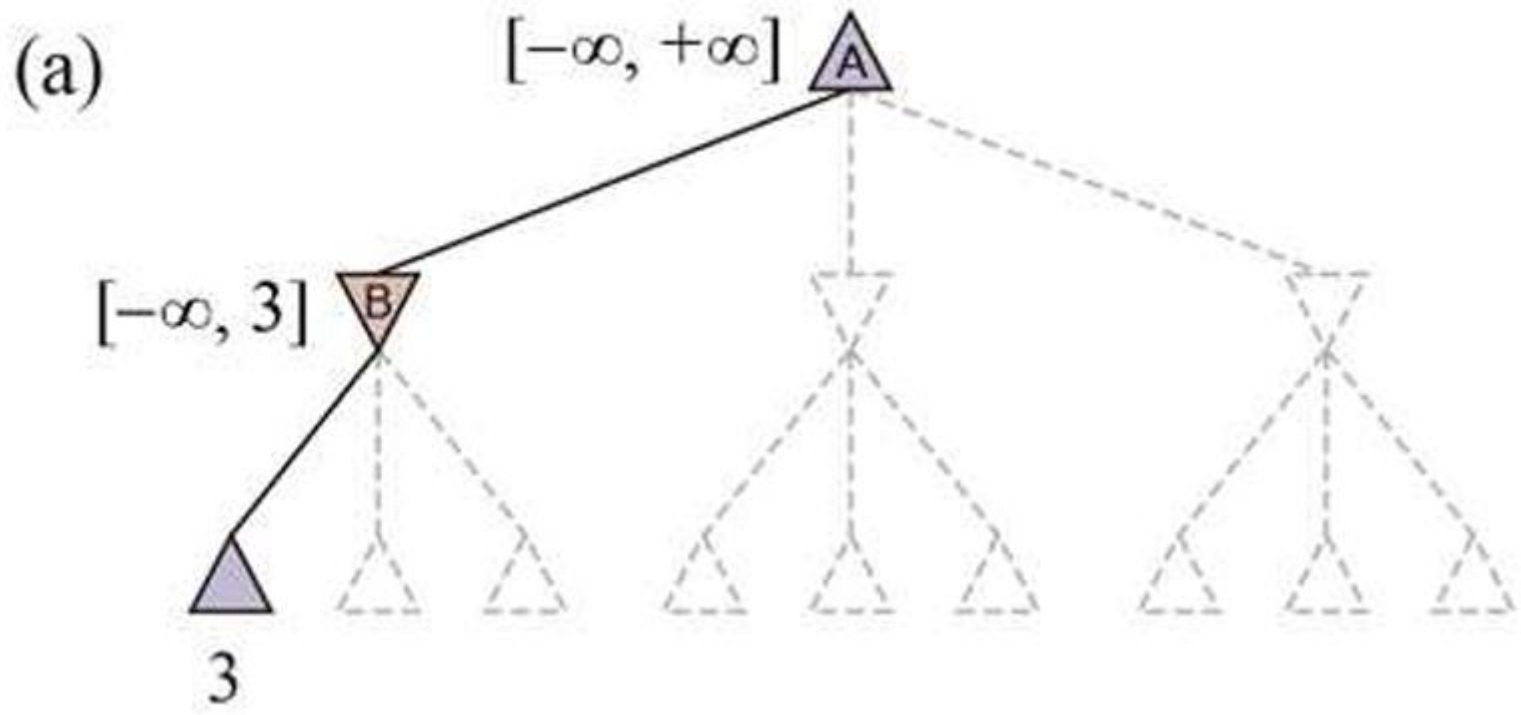
Given the complexity of DFS, you get 35^{80} states to visit

How to reduce the practical complexity of minimax?

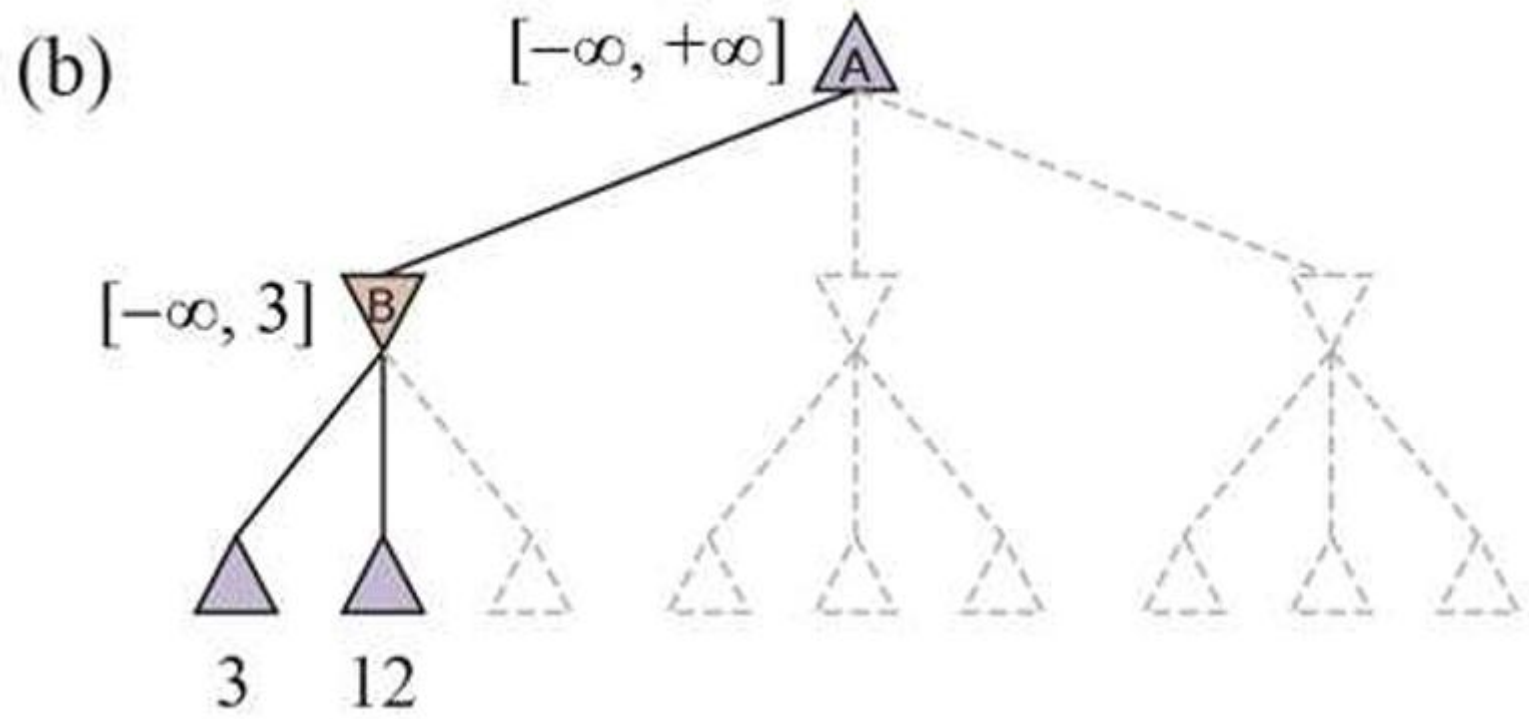
- Pruning!

$\alpha-\beta$ pruning: how to think about it

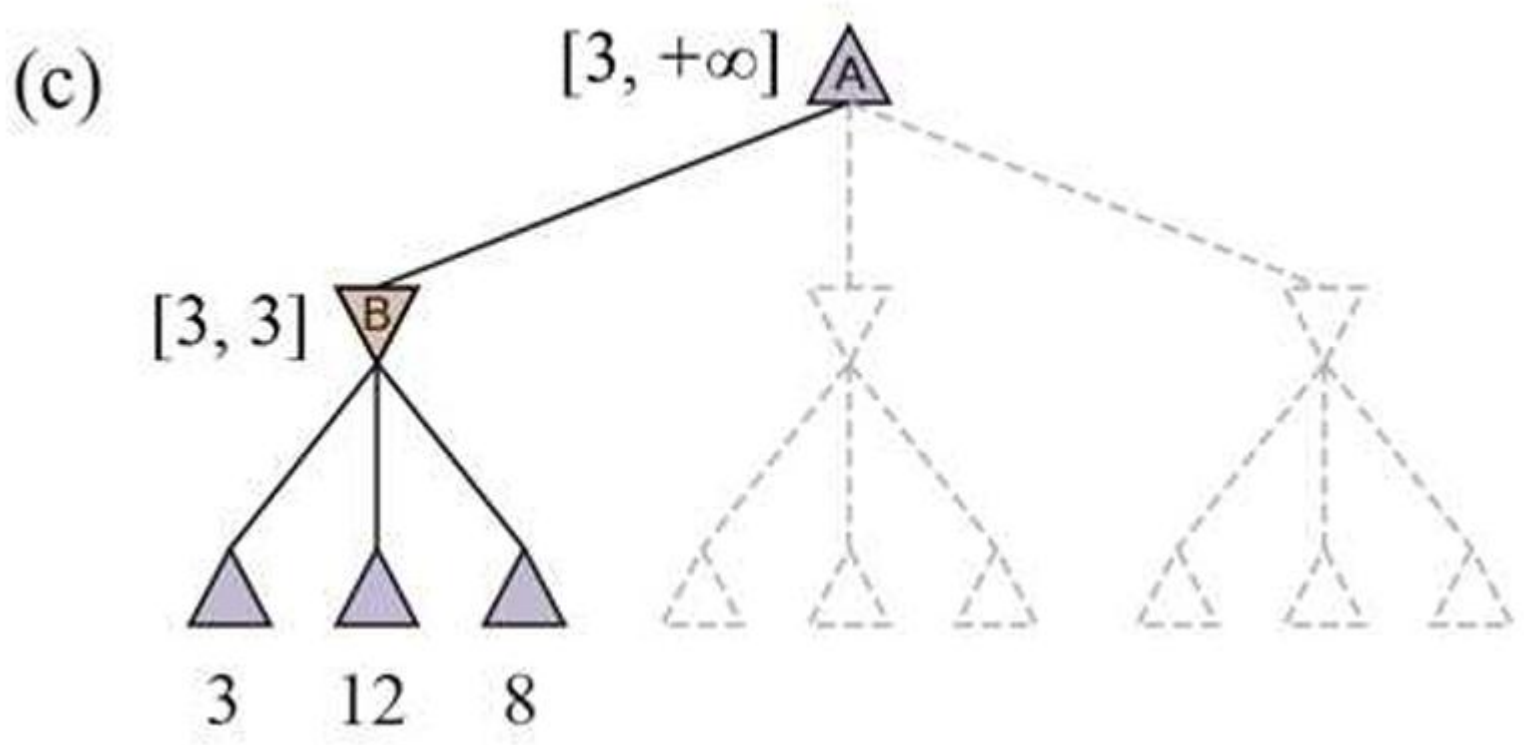
[min value, max value] notation



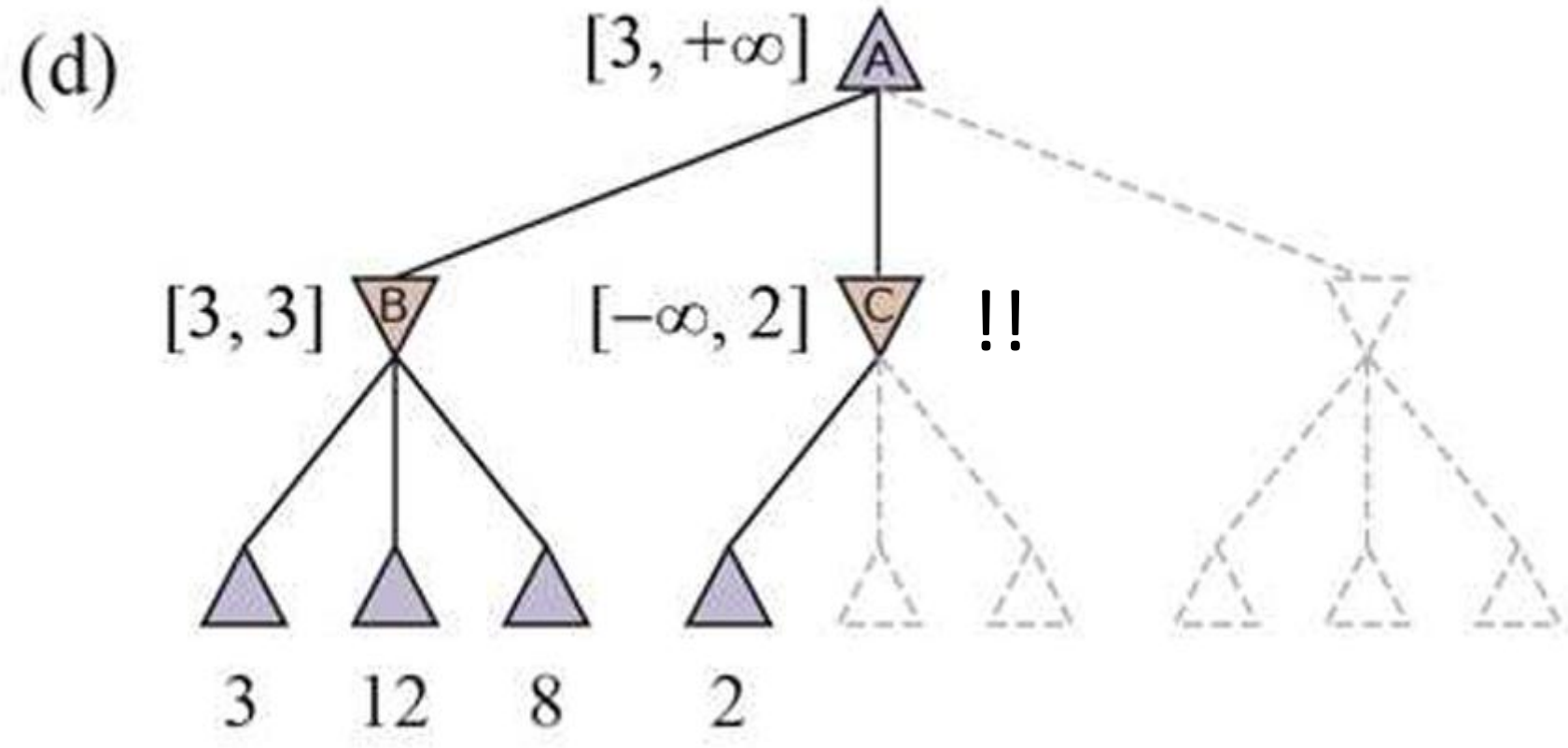
$\alpha-\beta$ pruning: how to think about it



$\alpha-\beta$ pruning: how to think about it



$\alpha-\beta$ pruning: how to think about it



```
function alphabeta(node, depth, α, β, maximizingPlayer) is
    if depth == 0 or node is terminal then
        return the heuristic value of node
    if maximizingPlayer then
        value := −∞
        for each child of node do
            value := max(value, alphabeta(child, depth − 1, α, β, FALSE))
            if value ≥ β then
                break (* β cutoff *)
            α := max(α, value)
        return value
    else
        value := +∞
        for each child of node do
            value := min(value, alphabeta(child, depth − 1, α, β, TRUE))
            if value ≤ α then
                break (* α cutoff *)
            β := min(β, value)
        return value
```

This time from Wikipedia: it also features *depth-limited search*

Choose the action that maximizes the initial call to *alphabeta(initialnode, depthlimit, -∞, +∞, True)*

α–β pruning

Why pruning?

It does not affect the solution

With **best** pruning, you can reduce time complexity to $O\left(b^{\frac{m}{2}}\right)$

- Good, but still too much for reasonable-sized problems ($\sqrt{b}$ effective branching factor)
- Best pruning depends on the **order of the moves**
- Generating moves that allows to prune the tree first leads to best pruning
- We won't go into details

Two general search strategies for games

Type A strategy

- Fully explore the search space up to a fixed depth
- Use heuristic function to evaluate utility for the nodes at the final depth (if not terminal)

Type B strategy

- Do not explore moves that look bad according to a given heuristic
- Follow promising moves as far as possible (possibly, up to the end of the game)

Type A works if the branching factor is not «very» large
Type B is useful when the branching factor is very large

Heuristics are either **learned** or exploit a weighted combination of game features

Horizon effect: the heuristic evaluates the position as good, but exploring one level more might change the evaluation

Monte-Carlo Tree Search (MCTS)

Alternative to alpha-beta

- Useful when branching factor is large (even with pruning, you would be limited to small depths)
- Useful when good evaluation functions are not available (but you can plug one in if you have it)

Rollout/Playout: from a given state, play out an entire episode until the end and get the utility.

Playout policy = function that decides which move to take at each step

- This is the part where reinforcement learning helped a lot (e.g., AlphaGo, AlphaZero, self-play)
- The rollout policy is **not (!)** the whole story

Selection policy = function that selects a path in the existing search tree

- Exploration (playout from less visited states) vs. exploitation (playout from frequently visited states)

Pure Monte-Carlo search: given a state, compute its value by performing N rollouts

- No search tree needed

MCTS steps

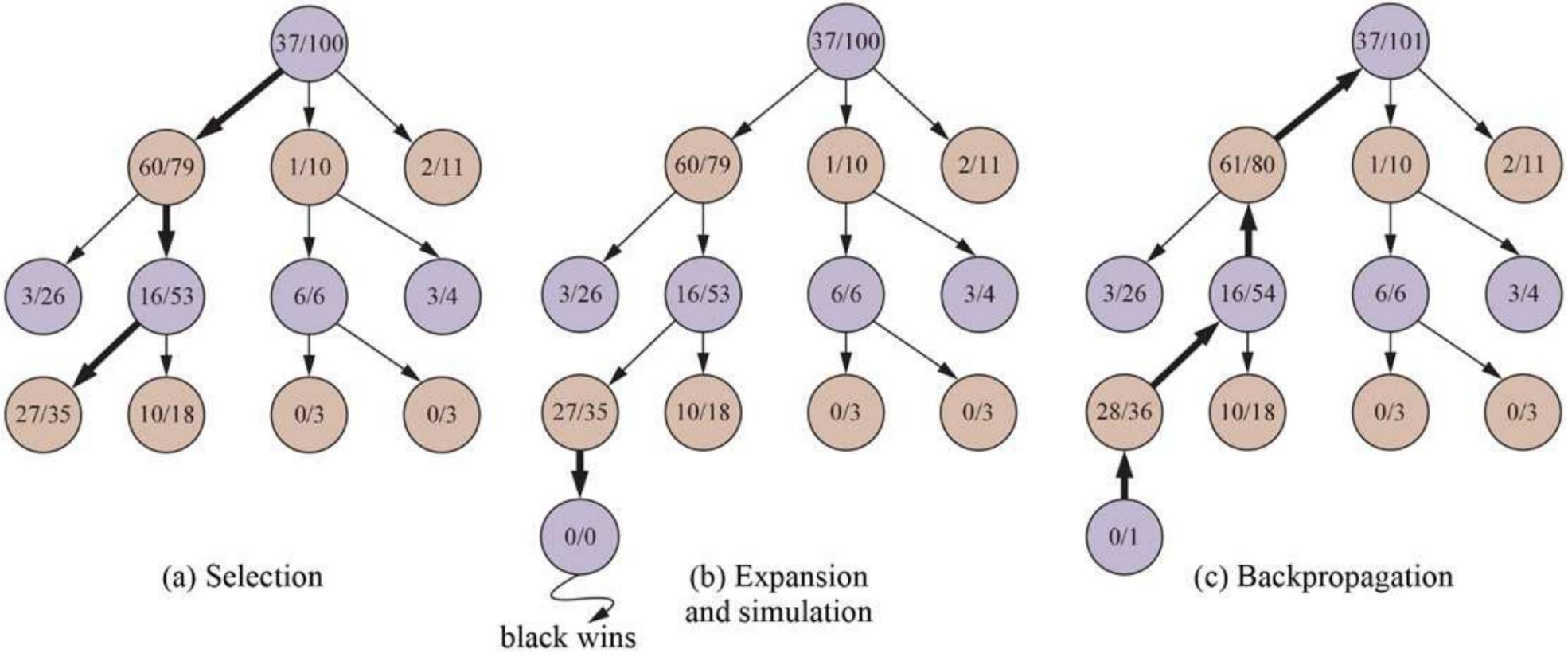
The value of a node in the search tree is its **expected utility**

- How much utility it accumulated over its playouts / number of playouts passing from that node

At the beginning, the search tree will contain only the initial state

Repeat for an allotted amount of steps:

- **Selection:** From the root node, use the selection policy to select a path down to a leaf
- **Expansion**: use the playout policy to generate one new node from the current leaf
- **Simulation:** perform one rollout from the newly expanded node using the playout policy. The search tree is not updated.
- **Back-propagation** (not that one): update the value of the nodes in the search tree from the newly added node up to the root, following the selection path


37/100
60/79
1/10
2/11
3/26
16/53
6/6
3/4
27/35
10/18
0/3
0/3
(a) Selection
37/100
60/79
1/10
2/11
3/26
16/53
6/6
3/4
27/35
10/18
0/3
0/3
0/0
black wins
(b) Expansion
and simulation
37/101
61/80
1/10
2/11
3/26
16/54
6/6
3/4
28/36
10/18
0/3
0/3
0/1
(c) Backpropagation

UCT: Upper Confidence bounds applied to Trees

Which selection policy? → follow the node that maximizes UCB1

During MCTS, compute Upper Confidence Bound (**UCB1**) for each node

$$UCB1(n) = \frac{U(n)}{N(n)} + C\sqrt{\frac{logN(Parent(n))}{N(n)}}$$

- N(n) = #playouts through node n
- U(n) = total utility (sum) of playouts through node n

$\frac{U(n)}{N(n)}$ = average utility = exploitation term

$\sqrt{\frac{logN(Parent(n))}{N(n)}}$ = high when N(n) low = high for rarely-explored nodes = exploration term

C balances exploration and exploitation

What happens to exploration over time?

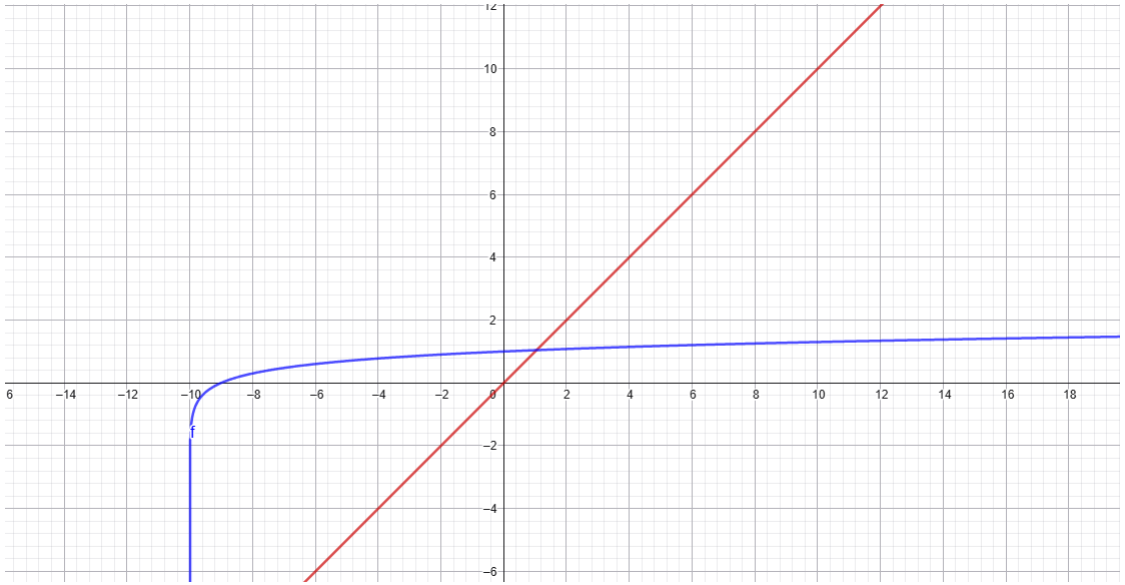
$$\sqrt{\frac{logN(Parent(n))}{N(n)}}$$

- N(Parent(n)) $\geq$ N(n)
  as every time you visit n you visit Parent(n)
- log(N(Parent(n))) $\leq$ N(n)
  from a certain number of playouts on

Suppose that Parent(n) received 10 playouts more than n

So, exploration term will go to zero as the number of playouts increases

- You don't want to explore forever



MCTS with UCT

Embarassingly parallel!

function MONTE-CARLO-TREE-SEARCH(state) **returns** an action
    tree ← NODE(state)
    while IS-TIME-REMAINING() **do**
        leaf ← SELECT(tree)
        child ← EXPAND(leaf)
        result ← SIMULATE(child)
        BACK-PROPAGATE(result, child)
    return the move in ACTIONS(state) whose node has highest number of playouts

MCTS with UCT

```
function MONTE-CARLO-TREE-SEARCH(state) returns an action
    tree ← NODE(state)
    while IS-TIME-REMAINING() do
        leaf ← SELECT(tree)
        child ← EXPAND(leaf)
        result ← SIMULATE(child)
        BACK-PROPAGATE(result, child)
    return the move in ACTIONS(state) whose node has highest number of playouts
```

The way UCT is designed, the action with highest #playouts is **almost** always the action with highest utility (highest win percentage)

Choosing the action based on utility only may select apparently good but severly underexplored actions (e.g., 2/3 vs. 65/100)

- high uncertainty!

Deterministic games

MCTS does not require heuristics…
- You can use it on any game if you just know the rules
- but you can use heuristics if you have some (e.g., to improve the policy)

MCTS relies on exploration, so it might take time to determine the value of an obviously good/bad move.
- Heuristics can also help here

MCTS with learned policy/heuristics consistently achieves super-human performance
- Chess, Go, Checkers, …

Take away: modern approaches combine search and learning, and more!

Published: 27 January 2016

Mastering the game of Go with deep neural networks and tree search

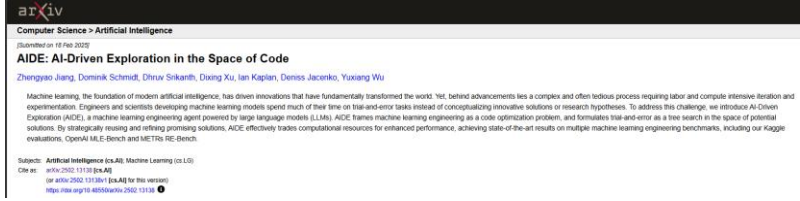
David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel & Demis Hassabis

Nature **529**, 484–489 (2016) | Cite this article

425k Accesses | **6479** Citations | **3100** Altmetric | Metrics

Abstract

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.



Combining LLMs with tree search

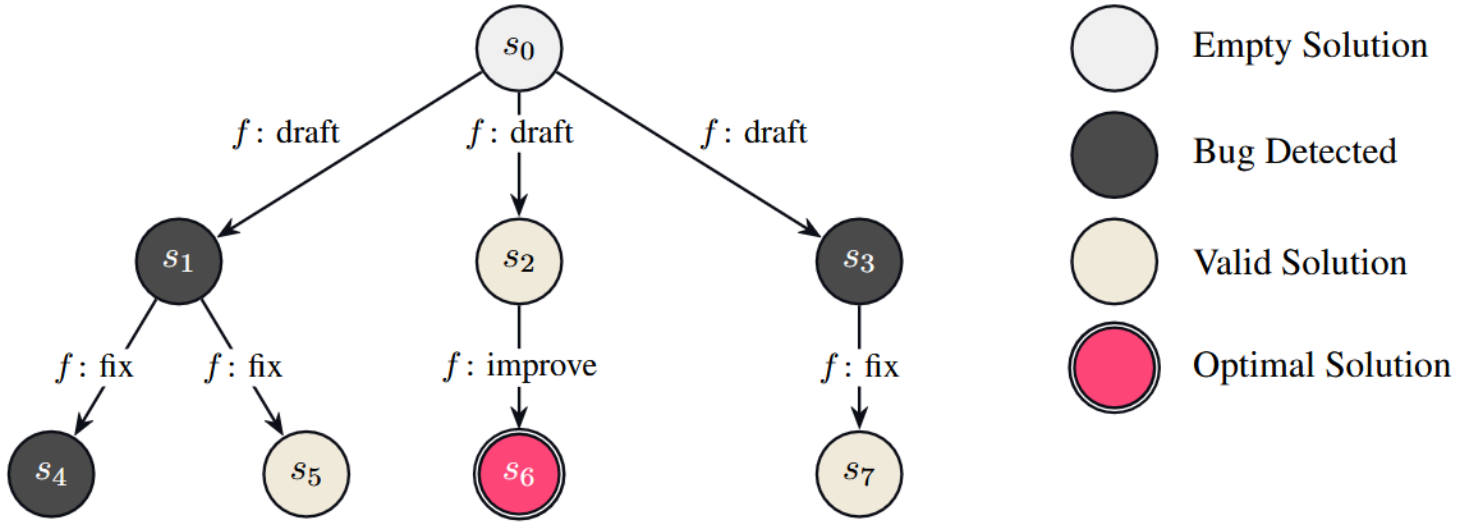
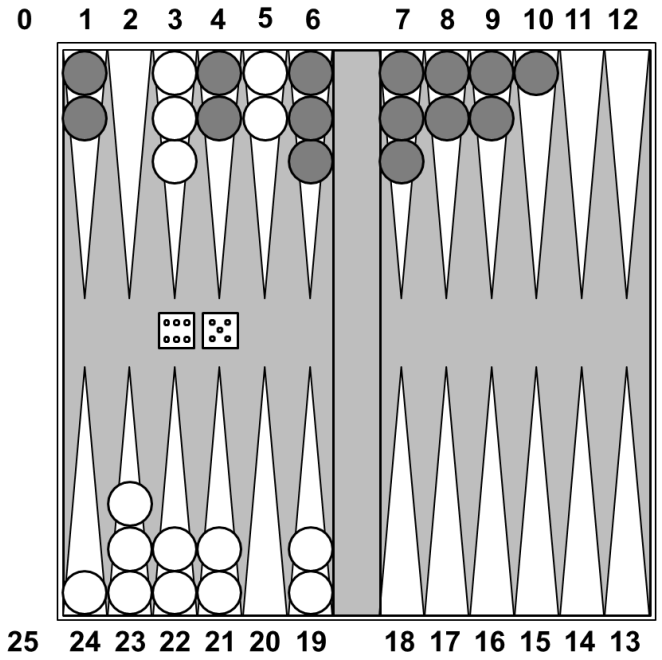


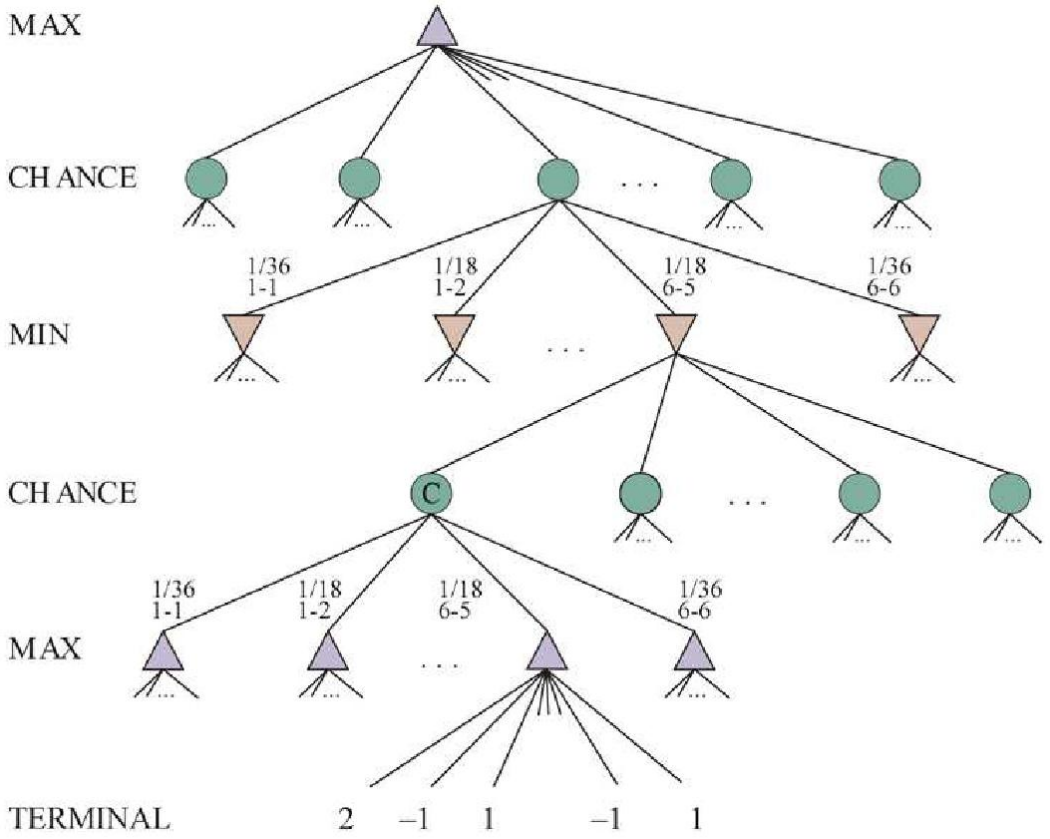
Figure 1: A sample solution tree T for AIDE, where each node is a Python script. Arrows represent transitions proposed by the coding operator f. Some branches terminate in a bug, while others lead to improved or optimal solutions.

https://arxiv.org/abs/2502.13138

Extensions to **stochastic** games

Chance node representing all possible outcomes (e.g., the result of a dice throw)





Expected Minimax

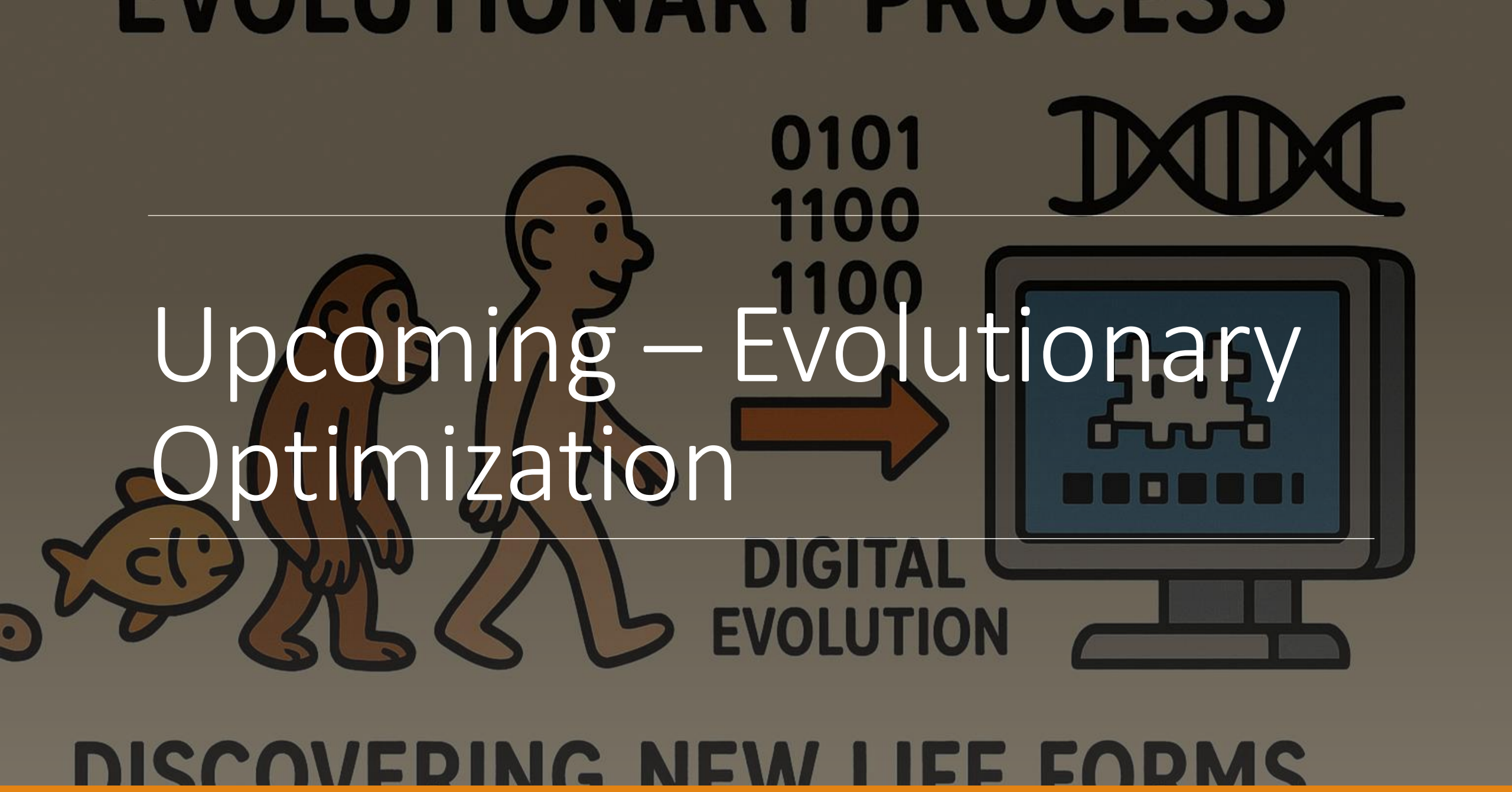
Same as minimax but you have to handle chance nodes as well

The value of a chance node is the expected value of all possible outcomes of the node

$$\text{ExpectedMINIMAX(s)} = \sum_r P(r) ExpectedMINIMAX(Result(s, r))$$

You can apply a similar reasoning on games with **partial information**, either deterministic (e.g., blind chess) or stochastic (e.g. poker)

- Ad-hoc adjustments are preferrable, we will not go in there



Upcoming – Evolutionary Optimization